ForgeAI

Use Cases & End-to-End Walkthroughs

Four realistic, complex scenarios showing exactly how a natural-language request flows through ForgeAI's multi-agent pipeline — and how the self-improving layer turns each run into measurable, compounding value.

What's inside

12 phases · 310 tests · runs fully offline · local models, no API keys

How a request flows (the model behind every example)

You submit one natural-language request. A **Manager** classifies and delegates it; an explicit workflow graph then sequences the specialists. Every agent reads from and writes to one shared *ProjectState* — agents never call each other directly.

```
START -> manager(intake) -> planner -> research -> memory -> coder
      -> execute(sandbox) -> tests -> review -> [approved?]
                                          | no  -> reflection -> coder   (retry)
                                          | yes -> git(propose PR) -> final -> END
```

And then — the part most tools skip:

When the run finishes, the **Evaluation Engine** scores it (outcome + cost + which prompt versions were used), stores it in the **Performance Database**, and updates the **Analytics** dashboard. Failures are remembered. Benchmarks track each release. That is what lets ForgeAI improve across runs without code changes.

EXAMPLE 1

Add JWT authentication to a backend API

Who: a developer with a FastAPI app and no auth yet. **Goal:** protected routes with login/signup, reviewed and on a PR.

The request

```
POST /agents/run
{ "user_request": "Add JWT authentication", "project_id": "auth-1" }
```

What happens, stage by stage

Agent / stage	What happens	Output / signal
Manager	Reads the request; **Dynamic Selection** classifies it as a **backend** task (matched signals: *auth, jwt, api*) and records the rationale.	task_type = backend
Planner	Breaks it into ordered tasks: add dependency, hash passwords, issue/verify tokens, protect routes, register/login endpoints, tests.	6 tasks
Researcher	Pulls only the relevant context — existing routes, settings, the password model.	scoped context
Memory	Recalls past decisions (e.g. 'we use argon2', 'tokens expire in 1440m') from prior runs.	long-term recall
Coder	Writes the auth module, token utils, and the protected dependency from the given context.	generated files
Execution	Installs deps and runs the build inside an isolated Docker sandbox.	build logs
Testing	Runs the suite; reports pass/fail with evidence.	tests passed
Review	Checks security, naming, structure → APPROVED, or requests changes (→ Reflection).	verdict: approved
Git	Authors a conventional commit on a local clone and **proposes** a PR — writes nothing yet.	PR proposal (gated)

You approve

The proposal appears in the **Approval Center**. You review the diff and click **Approve** → the PR is created on the real repository. Nothing reached GitHub without your decision.

What the self-improving layer recorded:

- an **Evaluation**: success=true, score=0.70, prompt_versions={planner: v1, coder: v1, …}
- the run now counts toward per-agent stats and the backend task-type history
- if the PR is later merged, that becomes a positive *pr_accepted* signal on the record

EXAMPLE 2

Fix a failing CI build — self-correction + failure memory

Who: a team whose pipeline broke. **Goal:** diagnose and fix, and never waste time re-diagnosing the same error twice.

First encounter — diagnose from scratch

Agent / stage	What happens	Output / signal
Coder → Execute	Code runs in the sandbox; the build fails: `ModuleNotFoundError: No module named 'jwt'`.	exit 1
Testing	Tests fail because the import is missing.	test_passed = false
Review	Not approved → routes to Reflection (the retry loop).	changes requested
Reflection	Diagnoses the root cause and proposes a concrete fix: *install pyjwt*. **Stores it** in the Failure Knowledge Base, keyed by a normalized signature.	fix proposed + stored
Coder (retry)	Applies the fix; Execute + Tests now pass; Review approves.	approved

Next encounter — fixed instantly

Weeks later, a different project throws the same error — buried in a long traceback. The signature normalizer collapses both to the same key, so Reflection **recalls the known-good fix** and applies it *without calling the model at all*.

```
error_signature('Traceback ...\nModuleNotFoundError: No module named \'jwt\'')
   ->  'modulenotfounderror:jwt'      # same key as the first time
recall('modulenotfounderror:jwt')
   ->  fix='pip install pyjwt'  outcome=RESOLVED   # reused, no LLM call
```

Why this matters: the system behaves like a Stack Overflow for itself. A fix that later fails is demoted, so the knowledge base self-corrects — no fix is trusted forever.

EXAMPLE 3

Build a CRUD REST API — multi-agent debate in action

Who: a startup that needs an endpoint fast but done right. **Goal:** a well-scoped plan chosen from competing approaches.

The request

```
POST /agents/run
{ "user_request": "Create a CRUD REST API for todo items",
  "project_id": "todos-1" }      # workflow built with debate_planner = 3
```

The Planner debates instead of guessing once

With debate enabled, **three independent Planner attempts** run from different angles — no attempt sees another's output:

Agent / stage	What happens	Output / signal
Attempt 0	*Simplest-increment* angle: ship a minimal create+list first.	plan A
Attempt 1	*Risk-first* angle: tackle validation, pagination, and error paths early.	plan B
Attempt 2	*Thorough* angle: cover tests, docs, and error handling explicitly.	plan C
Judge (Review)	Scores the candidates by a documented, deterministic rubric and records **which won and why**. The winning plan drives the rest of the pipeline.	winner + rationale

Then the normal pipeline runs on the winning plan

Researcher → Memory → Coder → Execute → Tests → Review → Git, exactly as in Example 1 — but starting from a plan chosen out of three, which widens the solution space and reduces the chance of a weak first guess derailing the run.

Recorded for the dashboard: the debate decision (winner index + judge rationale) lives on the run's audit trail; the Evaluation captures the score so debated vs. non-debated runs can be compared over time. Debate is **off by default** and deterministic — same input, same winner — so it is safe and reproducible.

EXAMPLE 4

A team that gets better over 100 runs

Who: any team running ForgeAI continuously. **Goal:** improvement that compounds — without anyone editing agent code.

Run 1 vs. Run 100

Agent / stage	What happens	Output / signal
Run 1	Baseline. Scored and stored. Prompts at v1. Failures start filling the knowledge base.	score recorded
Runs 2–50	Recurring errors are fixed instantly from memory. Each run's score + prompt version + outcome accrue in the Performance Database.	stats accumulate
Prompt A/B	Someone registers Planner **v2**. Runs split across v1/v2; per-version stats are derived from the records.	v1 vs v2 on the dashboard
Promotion gate	Once v2 has enough samples *and* clears the score margin, the system **recommends** promoting it — but it requires human approval; it never auto-promotes.	gated recommendation
Workflow-opt	If a task type succeeds uniformly without a step mattering, the system **suggests** A/B-testing the pipeline without it — advisory only, the graph is never auto-edited.	gated suggestion
Benchmarks	Each release runs the versioned scenario suite; the pass-rate trend shows whether the platform actually got better release over release.	trend per version

Why 100 > 1

The compounding effect: measurement (every run scored) + memory (failures reused) + comparison (versioned prompts & benchmarks) + gated learning (promote/optimize on real data) means Run 100 is measurably stronger than Run 1 — and you can *prove* it on the Analytics dashboard, not just claim it.

Try the whole loop yourself

```
# offline, deterministic, no models needed — every Phase 12 component:
cd apps/api
PYTHONPATH="$PWD:$PWD/../../packages" \
  uv run python ../../scripts/demo_phase12.py
```

All 12 phases complete · 310 tests passing · fully documented · runs on local models.